

This code implements a **Discrete Event Market Microstructure Simulator**. It models how different types of traders (Market Makers, Informed, Noise, and Momentum) interact with a **Limit Order Book (LOB)** across various asset classes like Equities, Commodities, and Crypto.

1. Foundation: Asset and Trader Modeling

The first section defines the "rules" for the assets and the "behaviors" of the agents.

Asset Class

This class uses a dataclass to store properties like `tick_size` (the minimum price increment) and volatility.

```
@dataclass
class Asset:
    name: str
    asset_type: AssetType
    tick_size: float
    initial_price: float
    # ...
    def __post_init__(self):
        """Initialize asset-specific parameters"""
        if self.asset_type == AssetType.EQUITY:
            self.depth_scale *= 2
        # ... logic to adjust spreads and volatility based on asset type
```

- **Explanation:** The `__post_init__` method ensures that different asset types behave realistically. For example, **Crypto** is initialized with higher volatility, while **Equities** are given deeper liquidity (`depth_scale`).

Trader Class

Traders are the "agents" in this system. Each is assigned a `TraderType`.

```
def decide_action(self) -> Tuple[str, float, Optional[float]]:
    actions = ['limit_bid', 'limit_ask', 'market', 'cancel']
    weights = [0.3, 0.3, 0.3, 0.1]
    action = np.random.choice(actions, p=weights)
    # ... logic for size and price offset
```

- **Explanation:** This method dictates the "intelligence" of the trader. They don't just buy or sell; they choose between adding liquidity (**Limit Orders**) or taking liquidity (**Market Orders**).

2. The Core Engine: The Limit Order Book (LOB)

This is the most technically significant part of the code. It manages the queue of buyers and sellers.

Efficient Data Management with Heaps

The LimitOrderBook class uses **Heaps** (priority queues) to ensure that the "Best Bid" and "Best Ask" are always at the top of the pile.

```
class LimitOrderBook:
    def __init__(self, asset: Asset):
        self.bids = [] # Max-heap for bids
        self.asks = [] # Min-heap for asks
        self.orders = {} # order_id -> LimitOrder lookup
```

- **Explanation:** * **Bids** are stored in a **Max-Heap** because the highest buyer has priority.
 - **Asks** are stored in a **Min-Heap** because the lowest seller has priority.
 - This allows for $O(1)$ access to the best prices and $O(\log n)$ for inserting new orders.

Market Order Matching

The match_market_order method simulates the "execution" of a trade.

```
def match_market_order(self, is_buy: bool, quantity: float, timestamp: datetime):
    # ...
    while quantity > 0 and self.asks:
        best_ask = self.asks[0]
        fill_qty = min(quantity, best_ask.remaining_quantity)
        total_cost += fill_qty * best_ask.price
        # ... update quantities and pop if filled
```

- **Explanation:** When a Market Buy arrives, it "walks the book," eating up the cheapest available Limit Ask orders until the buyer's quantity is satisfied or the book is empty.

3. Simulation Logic: The Market Pulse

The MarketSimulation class acts as the "clock" and the "environment" where traders and order books meet.

Poisson Arrival Process

Real markets don't have traders acting at perfectly fixed intervals. The code uses a **Poisson distribution** to model random arrivals.

```
for trader in self.traders:
    # Poisson arrival logic
    if np.random.poisson(trader.arrival_rate * self.time_step / 3600) > 0:
        # Trader chooses an asset and takes action...
```

- **Explanation:** This creates a stochastic (random) environment. Some moments are quiet, while others have "bursts" of high activity, mimicking real-world trading sessions.

4. Analytics and Metrics

The simulation tracks several key Market Microstructure variables:

Metric	Mathematical Formula	Description
Mid-Price	$P_{mid} = (P_{ask} + P_{bid}) / 2$	The fair value between the best buy and sell.
Spread	$S = P_{ask} - P_{bid}$	The cost of immediate execution (liquidity cost).
Volatility	$\sigma = \text{std}(R) \times \sqrt{T}$	The standard deviation of returns, scaled by time.

5. Visualization Suite

The code concludes with functions using matplotlib to render:

- **Depth Charts:** A "mountain" plot showing the volume of limit orders at various price levels.
- **Spread Convergence:** Shows how competition between Market Makers narrows the gap between buyers and sellers over time.
- **Price Evolution:** A time-series plot of the asset's price movement influenced by the trades.